

MÓDULO CARREGÁVEL DE KERNEL (KLD)

KLDs costumam ter sempre essas duas coisas:

- 1. Função *module event handler*
- 2. Macro *DECLARE_MODULE*

Função *module event handler*

Função que gerencia a inicialização e o encerramento de um KLD.

- O protótipo da função está em **<sys/module.h>**:

```
typedef int (*modeventhand_t)(module_t, int /* modeventtype_t */, void *);
```

- O tipo de evento está em **<sys/module.h>**:

```
typedef enum modeventtype {
    MOD_LOAD,      /* Set when module is loaded. */
    MOD_UNLOAD,    /* Set when module is unloaded. */
    MOD_SHUTDOWN,  /* Set on shutdown. */
    MOD_QUIESCE    /* Set when module is about to be unloaded. */
} modeventtype_t;
```

- Geralmente esse tipo de evento será usado em um switch-case com código para tratar cada possibilidade:

```
static int
modevent(module_t mod __unused, int event, void *arg __unused) {
    int error = 0;

    switch (event) {
        case MOD_LOAD:
            uprinf("Hello, world!\n");
            break;
        case MOD_UNLOAD:
            uprinf("Good-bye, cruel world!\n");
            break;
        default:
            error = EOPNOTSUPP; // Error: operation not supported
            break;
    }

    return (error);
}
```

Macro *DECLARE_MODULE*

Macro que registra um KLD e sua função *module event handler* no sistema.

- Protótipo da macro:

```
#include <sys/param.h>
#include <sys/kernel.h>
#include <sys/module.h>

DECLARE_MODULE(name, moduledata_t data, sub, order);
```

- name:

Nome do KLD.

- data (struct definida em **<sys/module.h>**):

```
typedef struct moduledata {
    const char      *name; // Nome oficial do módulo
    modeventhand_t  evhand; // Module event handler
    void            *priv; // Ponteiro para dados privados, se existirem
} moduledata_t;
```

- sub (especifica o subsistema do kernel ao qual o KLD pertence; enum definida em **<sys/kernel.h>**):

```
enum sysinit_sub_id {
    SI_SUB_DUMMY      = 0x0000000, /* Not executed. */
    SI_SUB_DONE       = 0x0000001, /* Processed. */
    SI_SUB_TUNABLES   = 0x0700000, /* Tunable values. */
    SI_SUB_COPYRIGHT  = 0x0800001, /* First console use. */
    SI_SUB_SETTINGS   = 0x0880000, /* Check settings. */
    SI_SUB_MTX_POOL_STATIC = 0x0900000, /* Static mutex pool. */
    SI_SUB_LOCKMGR    = 0x0980000, /* Lock manager. */
    SI_SUB_VM         = 0x1000000, /* Virtual memory. */
    ...
    SI_SUB_DRIVERS     = 0x3100000, /* Device drivers. */
    ...
};
```

Geralmente usamos **SI_SUB_DRIVERS** para o subsistema de driver de dispositivo.

- order (ordem de inicialização do KLD dentro do subsistema sub; enum definida em **<sys/kernel.h>**):

```
enum sysinit_elem_order {
    SI_ORDER_FIRST  = 0x0000000, /* First. */
    SI_ORDER_SECOND = 0x0000001, /* Second. */
    SI_ORDER_THIRD  = 0x0000002, /* Third. */
    SI_ORDER_FOURTH = 0x0000003, /* Fourth. */
    SI_ORDER_MIDDLE = 0x1000000, /* Somewhere in the middle. */
    SI_ORDER_ANY    = 0xffffffff /* Last. */
};
```

Geralmente usamos **SI_ORDER_MIDDLE**.

Compilação e carregamento

- Crie um makefile:

```
KMOD=hello
SRCS=hello.c

.include <bsd.kmod.mk>
```

- Compile o KLD:

```
>>> make
O executável hello.ko será criado.
```

- Carregue o KLD:

```
>>> sudo kldload ./hello.ko
>>> sudo make load
```

- Descarregue o KLD:

```
>>> sudo kldunload ./hello.ko
>>> sudo make unload
```

- Para ver os status de todos os arquivos dinamicamente linkados ao kernel:

```
>>> kldstat
```

Character drivers costumam ter sempre essas três coisas

1. Funções *d_foo*
2. Character device switch table
3. Funções *make_dev* e *destroy_dev*

Funções *d_foo*

Operações de I/O que um processo pode executar em um dispositivo, geralmente associadas ao sistema de arquivos.

- Os protótipos das funções estão em **<sys/conf.h>**.

- As funções possíveis são:

Função	Descrição
d_open	Called to open the device in preparation for I/O operations
d_close	Called to close the device
d_read	Called to read data from the device
d_write	Called to write data to the device
d_ioctl	Called to perform an operation other than a read or a write
d_poll	Called to check the device to see whether data is available for reading or space is available for writing
d_mmap	Called to map a device offset into a memory address
d_kqfilter	Called to register the device with a kernel event list
d_strategy	Called to start a read or write operation and then immediately return
d_dump	Called to write all physical memory to the device

Character device switch table

Struct que especifica quais funções *d_foo* um character driver implementa.

- A struct *cdevsw* está definida em **<sys/conf.h>**:

```
struct cdevsw {
    int      d_version;
    u_int    d_flags;
    const char *d_name;
    d_open_t *d_open;
    d_fdopen_t *d_fdopen;
    d_close_t *d_close;
    d_read_t *d_read;
    d_write_t *d_write;
    d_ioctl_t *d_ioctl;
    d_poll_t *d_poll;
    d_mmap_t *d_mmap;
    d_strategy_t *d_strategy;
    dumper_t *d_dump;
    d_kqfilter_t *d_kqfilter;
    d_purge_t *d_purge;
    d_spare2_t *d_spare2;
    uid_t     d_uid;
    gid_t     d_gid;
    mode_t    d_mode;
    const char *d_kind;
    /* These fields should not be messed with by drivers. */
    LIST_ENTRY(cdevsw) d_list;
    LIST_HEAD(, cdev) d_devs;
    int         d_spare3;
    struct cdevsw *d_gianttrick;
};
```

Somente **d_version** e **d_name** são obrigatórios. Geralmente atribui-se **D_VERSION** à versão, que é substituída pela versão na qual o FreeBSD foi compilado.

Funções *make_dev* e *destroy_dev*

A função *make_dev* é responsável por pegar uma *cdevsw* e criar um nó de character device em */dev*.

- Protótipo de *make_dev*:

```
#include <sys/param.h>
#include <sys/conf.h>

struct cdev *
make_dev(struct cdevsw *cdevsw, int minor, uid_t uid, gid_t gid,
         int perms, const char *fmt, ...);
```

A função *destroy_dev* é responsável por pegar a struct *cdev* retornada por *make_dev* e destruir o nó de character device.

- Protótipo de *destroy_dev*:

```
#include <sys/param.h>
#include <sys/conf.h>

void
destroy_dev(struct cdev *dev);
```

Macro *DEV_MODULE*

Macro para simplificar a criação de KLDs que são character drivers.

- O protótipo da macro está em **<sys/conf.h>**:

```
#define DEV_MODULE(name, evh, arg) \
static moduledata_t name##_mod = { \
    #name, \
    evh, \
    arg \
};
DECLARE_MODULE(name, name##_mod, SI_SUB_DRIVERS, SI_ORDER_MIDDLE)
```

Módulos desenvolvidos

- 01-hello/
Imprime uma mensagem ao ser carregado e outra ao ser descarregado.
-